

Option 3B design: per-process proxy settings

Written 2026-09-22, the dedicated design doc for option 3B of the four network-layer alternatives ranked in `microsoft-365-copilot-interception-feasibility.md`'s Reason 1 section — the option to check *first*, since if it applies it closes Reason 1 for zero new engineering. Companion to `option-4-process-aware-redirect-design.md`, the fallback if this one doesn't hold up.

The precedent this option generalizes from

The whiteboard/transcript source names Claude as the existing example: "*HTTP_PROXY & HTTPS_PROXY env vars per process (e.g. Claude in settings.json).*" Worth being precise about **why** that works, since the mechanism doesn't transfer just because the pattern name sounds generic. Claude Code is a cooperative, proxy-aware application: it reads its own proxy configuration from a file this codebase can write to (its `settings.json`), and Claude Code's own startup code then applies that as its process environment before making any network call. This is an **application-level feature Claude Code chose to support**, not an OS-provided mechanism for scoping a proxy to one named process. The open question for M365 Copilot is whether an equivalent, write-into-a-file cooperative surface exists — not whether Windows has a generic "set env var for this one EXE" primitive, because it doesn't (checked below).

Checkpoint, part 1: the classic Office networking stack is a dead end for this — confirmed, not assumed

Checked directly, not inferred from Reason 1's earlier finding alone:

- **Office's native networking (WINWORD.EXE and siblings) uses WinINET**, confirmed via Microsoft's own documentation ecosystem. WinINET proxy configuration is **per-user**, sourced from the registry / Internet Explorer settings — never from Unix-style `HTTP_PROXY` / `HTTPS_PROXY` environment variables, which WinINET-based Windows applications don't read at all.
- **WinHTTP** (the sibling API some Windows services use) is **per-machine**, not per-user and not per-process. Achieving true per-application scoping with WinHTTP would require the *application's own code* to call the per-session/per-request WinHTTP override APIs itself — again, cooperation the target app has to build in, not something available externally.
- **Neither stack exposes a native "scope this proxy setting to one named executable" primitive.** Windows proxy configuration is fundamentally per-user or per-machine at the OS

level; anything narrower has to come from the application itself choosing to read a narrower config surface.

This directly explains why 3B "already works for Claude" doesn't mechanically generalize:

Claude chose to support it. Nothing found so far suggests Office's native stack does the same, and this lines up with what Reason 1's own admin-doc research already established ("*No config file, registry key, environment variable, or admin policy... that repoints where any of this traffic goes*") — the same absence, confirmed from a different angle.

Checkpoint, part 2: a real, more promising lead — the Copilot pane is WebView2-hosted (on Windows only)

Before closing 3B entirely, one more angle: **the Copilot chat experience in**

Word/Excel/PowerPoint is rendered through WebView2. Sourcing note, stated plainly: the specific Microsoft KB article this traces to ("WebView2 Conflict in Office Applications") did not render its real content through two separate fetch attempts — both landed on a generic Microsoft 365 help hub page instead. What's cited here is **secondhand**, via BleepingComputer's reporting on that article ("*Excel, Word, PowerPoint, OneNote, Publisher, and Access*" named as affected, with "*Copilot, Share, and Room Finder*" named as the WebView2-dependent features), not a direct read of Microsoft's own page. Treat the Word/Excel/PowerPoint-Copilot link as well-supported, not KB-primary-sourced.

A separate, directly-read Microsoft Learn document changes the platform scope of this whole option, definitively: "*WebView2 Runtime isn't installed on devices running macOS.*" That's not an inference — it's Microsoft's own deployment documentation (`webview2-install`), stated outright. **3B is a Windows-only option, full stop.**

The same Microsoft Learn document also gives Outlook a real, if narrower, citation: it names "*Room Finder and Meeting Insights*" as WebView2-based Outlook features, and states "*you see multiple instances of Microsoft Edge WebView2 running under the Microsoft Outlook process*" — confirming Outlook hosts WebView2 instances routinely. It does not name Copilot specifically, so this is corroborating, not dispositive, for Outlook's Copilot pane.

WebView2 is Chromium-based, and Chromium has its own, real, **externally-injectable** proxy override that has nothing to do with WinINET/WinHTTP:

- `WEBVIEW2_ADDITIONAL_BROWSER_ARGUMENTS` — a real, Microsoft-documented environment variable. Setting it to `--proxy-server=<host>:<port>` passes that flag to the underlying Chromium browser process WebView2 launches. Confirmed via Microsoft's own WebView2 documentation (`webview-features-flags`), not a third-party claim.
- A registry alternative exists too (`HKCU\SOFTWARE\Policies\Microsoft\Edge\WebView2\AdditionalBrowserArguments`), but that's a

per-user **policy** key affecting every WebView2 host app for that user — not narrower than the env var, and not specific to Office or Copilot.

- Confirmed additive behavior: whatever the env var/registry specifies is **appended** to any browser arguments the host app (Office) already passes — so this doesn't require Office's cooperation or even its awareness. It layers on top.

Why this could actually be process-scoped, unlike anything on the classic stack:

environment variables are inherited per-process by ordinary OS rules. If this variable is set only in the target Office process's own environment block (not the user's or machine's), it would apply only to that process's WebView2/Chromium instances — a real, narrow scope, not a system-wide one. That's a materially different proposition from options 1 and 2, which have no such boundary at all.

"The target Office process" is not one process, and — since this option is Windows-only — it's a Windows process list specifically, not the cross-platform one Reason 1 deals with generally. On Windows, the candidates are `WINWORD.EXE` , `EXCEL.EXE` , `POWERPNT.EXE` , and `OUTLOOK.EXE` , each independently, at different confidence levels:

Process	Confidence Copilot's pane is WebView2-hosted
<code>WINWORD.EXE</code>	Directly confirmed — a Microsoft Q&A moderator response, in a real support thread about the Word Copilot pane specifically, states outright: " <i>Word's pane runs on WebView2</i> " (repairing "the Edge WebView2 Runtime (this is the 'web UI engine' Word uses)" was the given fix). First-party, not secondhand.
<code>EXCEL.EXE</code> , <code>POWERPNT.EXE</code>	Named directly, but secondhand-sourced (via BleepingComputer's reporting on the KB article, see above) — not yet independently confirmed the way Word now is
<code>OUTLOOK.EXE</code> (new Outlook)	Confirmed by architecture — the whole app is a WebView2 shell, Copilot included by construction
<code>OUTLOOK.EXE</code> (classic Outlook)	Circumstantial only — Outlook confirmed to host WebView2 instances (Room Finder, Meeting Insights), Copilot not named specifically

Whatever mechanism actually injects `WEBVIEW2_ADDITIONAL_BROWSER_ARGUMENTS` scoped to "the" target process needs to match against **all four**, independently — a user might have the Copilot pane open in Excel and Outlook simultaneously, and both need the same treatment. This also means the live check in item 1 below should be run against more than one host app before concluding the WebView2 lever works generally, and classic Outlook specifically needs its own check rather than an assumption it matches Word/Excel/PowerPoint.

Checkpoint, part 3: the macOS equivalent was checked too — closed by architecture, not just absent

Worth checking rather than assuming: does macOS have its *own* version of the same trick, using whatever Apple's equivalent to WebView2 is? Checked directly, and the answer is a firmer no than "the specific technology isn't there."

Mac Office's embedded-web mechanism is confirmed: a Microsoft Learn table (`browsers-used-by-office-web-add-ins`) states plainly — Windows uses "*Microsoft Edge (Chromium-based) with WebView2*," Mac uses "*Safari with WKWebView*." WKWebView is Apple's WebKit-based embedded browser control, the real macOS counterpart. (Scoped to "Add-ins" by title, not named for Copilot specifically — but WKWebView is essentially the only sanctioned embedded-browser engine available to any macOS app, first- or third-party, so it's a near-certainty Copilot's own pane uses the same thing.)

WKWebView has no equivalent to `WEBVIEW2_ADDITIONAL_BROWSER_ARGUMENTS` — not a differently-shaped one, none at all:

- Apple's only proxy-configuration surface for WKWebView is `WKWebsiteDataStore.proxyConfigurations` (the `ProxyConfiguration` API, introduced iOS 17.0) — a **programmatic API the host app's own code must call itself, before creating the WKWebView instance**. There is no environment variable, command-line flag, or registry/plist key that injects a proxy from outside the app the way the Windows env var does. Office would have to opt into this in its own code, for our purposes, which it has no reason to do.
- **WKWebView doesn't inherit the system-wide proxy setting either**, confirmed via Apple's own developer forums. So there isn't even a broader, "user-scoped, more blast radius, no kernel component" fallback the way there might have been on Windows — there's no system-level lever WKWebView reads at all, narrow or broad.

Net: this is closed by architecture, not merely by WebView2's absence. Even imagining a hypothetical macOS-native equivalent of the Windows mechanism, it doesn't exist — the only lever Apple provides requires the target application's cooperation, which is exactly the category of thing 3B was trying to avoid needing. Option 4 is the only real path for macOS, and this is why, not just that.

What's still open — two concrete items, not assumptions to build on yet

1. **Unconfirmed: does the actual Chathub WebSocket connection originate inside the WebView2/Chromium context, or from Office's native code (which then only uses the WebView2 pane to render/display)?** This is the load-bearing question. If the connection is genuinely made from Chromium's own network stack (plausible — a persistent chat pane is a common shape for "the native shell just hosts a page that does everything itself," and Copilot is explicitly named as one of the WebView2-hosted features), `--proxy-server` reaches it directly. If the SignalR/WebSocket call is actually made by Office's native code and only the *rendering* happens in WebView2, this option doesn't reach it at all, and the finding in part 1

(WinINET, no per-process lever) governs instead. **Cheap to settle:** set the env var scoped to a real Office process, drive a real Copilot chat interaction, and check whether the connection actually appears at the listener bound to that port — a live test, not further reading.

2. **Unconfirmed: the actual injection mechanism for a genuinely process-scoped (not user-scoped) environment variable.** Setting an env var only in one specific process's environment block, for a process this codebase doesn't launch itself (Office is typically already running, or launched by Explorer/a file association, not by `ai-protect`), needs its own real mechanism — a process-creation hook analogous in spirit to option 4's connection hook, just intercepting process *creation* instead of connection *establishment*. If that turns out to be as heavy as option 4's kernel work, the "3B is free" premise weakens; a fallback worth noting now is **user-scoped** injection (the env var or registry key set for the user, not the specific process) — broader than ideal (every WebView2-hosted app for that user, not just Office/Copilot), but still far narrower than options 1/2's whole-domain, every-process blast radius, and mechanically simple (an ordinary environment/registry write, no kernel component at all).

Where this leaves 3B

Not closed, not confirmed — genuinely open, pending the two live checks above, and now with a **defined boundary**: Windows only, never macOS, confirmed directly rather than assumed.

Meaningfully more promising than the transcript's original framing suggested it might be by default, because of the WebView2 angle — but for a reason specific to *how Copilot's UI happens to be built*, not because Windows offers a general per-process proxy primitive. If item 1 fails (the connection isn't Chromium-originated) for Word/Excel/PowerPoint, or if classic Outlook's Copilot pane turns out not to be WebView2-hosted at all, 3B is closed for that app for the same underlying reason Reason 1 was already closed at the app-config level — and option 4 (which does cover macOS) becomes the only path for whichever apps/platforms 3B doesn't reach.

Sources

- [WebView2 conflict in Office applications — Microsoft Support](#) — the specific known-issue page; **not independently read** (two fetch attempts both resolved to a generic help hub instead). Content cited here comes via [BleepingComputer's reporting on it](#), not a direct read
- [MS Word — the Copilot pane no longer allows highlighting text — Microsoft Q&A](#) — directly read; a Microsoft moderator response states outright that Word's Copilot pane "runs on WebView2." First-party confirmation specific to Word, independent of the KB article above
- [Microsoft Edge WebView2 and Microsoft 365 Apps — Microsoft Learn](#) — directly read; confirms WebView2 isn't installed on macOS, and names Outlook's Room Finder/Meeting Insights as WebView2-based

- [WebView2 browser flags — Microsoft Edge Developer documentation](#) — `WEBVIEW2_ADDITIONAL_BROWSER_ARGUMENTS` , the registry alternative, and the additive-append behavior
- [Setting WinINET Proxy Configurations in WinHTTP — Microsoft Learn](#) — WinINET (per-user) vs. WinHTTP (per-machine) scoping
- [Browsers and webview controls used by Office Add-ins — Microsoft Learn](#) — directly read; the Windows-WebView2 / Mac-WKWebView table
- [What is the recommended way to programmatically apply proxy to WKWebView — Apple Developer Forums](#) and [Use a HTTP Proxy with WKWebView — Apple Developer Forums](#) — the `WKWebsiteDataStore.proxyConfigurations` API being host-app-code-only, and WKWebView not inheriting the system proxy
- [microsoft-365-copilot-interception-feasibility.md](#) , Reason 1 — the original admin-doc finding this doc's part-1 checkpoint corroborates from a different angle